

Assignment-5

2. Find First and last position of a element in sorted array

34. Find First and Last Position of Element in Sorted Array Solved

Medium Topics Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:
Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:
Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:
Input: `nums = []`, `target = 0`
Output: `[-1,-1]`

23.4K 370 202 Online

```
Java  
33  
34     return last;  
35 }  
36  
37 public int[] searchRange(int[] nums, int target) {  
38     int[] ans = new int[2];  
39     ans[0] = firstOccurrence(nums, target);  
40     ans[1] = lastOccurrence(nums, target);  
41     return ans;  
}
```

Accepted 88 / 88 testcases passed

Raj Vardhan Singh Gangwar submitted at May 06, 2026 14:58

Runtime: 0 ms | Beats: 100.00%
Memory: 48.40 MB | Beats: 13.58%

3. Search Insert position

35. Search Insert Position Solved

Easy Topics Companies

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:
Input: `nums = [1,3,5,6]`, `target = 5`
Output: `2`

Example 2:
Input: `nums = [1,3,5,6]`, `target = 2`
Output: `1`

Example 3:
Input: `nums = [1,3,5,6]`, `target = 7`
Output: `4`

Constraints:

18.9K 442 193 Online

```
Java  
10  
11         } else {  
12             high = mid - 1;  
13         }  
14     }  
15     return low;  
16 }
```

Accepted 66 / 66 testcases passed

Raj Vardha... submitted at May 06, 2026 15:06

Runtime: 0 ms | Beats: 100.00%
Memory: 44.82 MB | Beats: 28.53%

4. Merge Sorted array

The screenshot shows the LeetCode interface for the "Merge Sorted Array" problem (88). The problem description states: "You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively. Merge `nums1` and `nums2` into a single array sorted in non-decreasing order. The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`."

Example 1:
Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`
Output: `[1,2,2,3,5,6]`
Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`. The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

Example 2:
Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`
Output: `[1]`

The code editor shows a Java solution using a two-pointer approach:

```
1 class Solution {
2     public void merge(int[] nums1, int m, int[] nums2, int n) {
3         int ans[] = new int[n + m];
4         int i = 0, j = 0, k = 0;
5         while (i < m && j < n) {
6             if (nums1[i] <= nums2[j]) {
7                 ans[k++] = nums1[i];
8                 i++;
9             } else if (nums1[i] > nums2[j]) {
10                ans[k++] = nums2[j];
11                j++;
12            }
13        }
14        while (i < m) {
15            ans[k++] = nums1[i];
16            i++;
17        }
18        while (j < n) {
19            ans[k++] = nums2[j];
20            j++;
21        }
22    }
23 }
```

The submission is accepted with a runtime of 77 ms (Beats 3.64%) and memory usage of 43.28 MB (Beats 98.38%).

5. Kth Largest Element in an Array

The screenshot shows the LeetCode interface for the "Kth Largest Element in an Array" problem (215). The problem description states: "Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array. Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element. Can you solve it without sorting?"

Example 1:
Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: `5`

Example 2:
Input: `nums = [3,2,3,1,2,4,5,6]`, `k = 4`
Output: `4`

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

The code editor shows a Java solution using a min-heap (PriorityQueue):

```
1 PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
2 for(int i=0; i<nums.length; i++){
3     pq.add(nums[i]);
4 }
5 while(k-1){
6     pq.poll();
7     k--;
8 }
9 return pq.poll();
```

The submission is accepted with a runtime of 103 ms (Beats 11.40%) and memory usage of 67.74 MB (Beats 82.48%).